

## 리전(Region) 이란?

AWS의 데이터 센터가 위치한 물리적으로 멀리 떨어진 지리적 위치(나라/도시 단위)로 AWS는 전 세계에 여러 리전을 두고 있다.

예: 서울(ap-northeast-2), 도쿄(ap-northeast-1), 버지니아(us-east-1)

## 가용 영역(AZ-Availability Zone)이란?

AWS 리전(region) 내에 있는 물리적으로 분리된 데이터 센터의 묶음

같은 리전에 속하지만 서로 다른 장소에 있는 독립된 데이터 센터들을 의미

예: 서울 리전(ap-northeast-2)에는

→ ap-northeast-2a, ap-northeast-2b, ap-northeast-2c, ap-northeast-2d 등 여러 AZ가 있음

예시(AZ는 내부적으로 독립된 전력, 네트워크, 냉각, 건물로 구성되어 있다)

```
Region: ap-northeast-2 (서울)
├─ AZ: ap-northeast-2a (서울 A센터)
├─ AZ: ap-northeast-2b (서울 B센터)
├─ AZ: ap-northeast-2c (서울 C센터)
└─ AZ: ap-northeast-2d (서울 D센터)
```

## 왜 가용 영역이 중요한가?

| 이유            | 설명                                              |
|---------------|-------------------------------------------------|
| ✅ 고가용성(HA) 확보 | 하나의 AZ에 장애가 발생해도, 다른 AZ에서 서비스 계속 가능             |
| 📦 이중화 구성      | DB는 Multi-AZ 구성으로 실시간 복제 가능                     |
| ☁ 탄력적 인프라     | EC2, RDS, ELB 등 대부분의 AWS 서비스는 AZ 단위로 배포 가능      |
| 🌐 단일 장애 지점 방지 | AZ 간 네트워크는 초고속으로 연결되지만 물리적 거리는 분리되어 있어 장애 격리 가능 |

## EC2(Elastic Compute Cloud)란?

EC2는 AWS가 제공하는 가상 서버(Virtual Machine) 서비스로 즉, AWS 클라우드 위에서 리눅스나 윈도우 기반 서버를 클릭 몇 번으로 만들 수 있게 해주는 서비스이다.

## AWS에서 프로비저닝(Provisioning)이란?

클라우드 자원(서버, 데이터베이스, 네트워크 등)을 필요에 따라 생성하고 설정하는 작업을 의미하는 것으로 쉽게 말해 "필요한 만큼 자원을 자동 또는 수동으로 준비하고 사용할 수 있게 만드는 것"을 뜻한다.

### 예시로 이해하기

#### 1. 전통적인 방식에서는

- 서버를 직접 구매하고,
- OS를 설치하고,
- 네트워크를 구성하고,
- 보안 설정도 직접 해야 했다.

#### 2. AWS에서 프로비저닝에서는

- EC2 인스턴스를 2개 생성하고,
- RDS(MySQL) 인스턴스를 하나 만들고,
- S3 버킷도 자동 생성,
- IAM 권한도 세팅

이 모든 과정을 클릭 몇 번 또는 코드 한 줄로 자동화 하는 과정을 프로비저닝이라 한다.

## VPC란?

AWS의 VPC (Virtual Private Cloud)는 AWS에서 사용자가 자신만의 가상 네트워크 환경을 구성할 수 있게 해주는 서비스이다. 마치 AWS 클라우드 내에 사용자 전용의 데이터센터를 만드는 것과 비슷하다.

- Virtual Private Cloud의 약자

- AWS 클라우드 안에 격리된 네트워크 공간
- 사용자가 직접 IP 주소 범위, 서브넷, 라우팅 테이블, 인터넷 게이트웨이, 보안 그룹 등을 설정할 수 있다.

## 왜 사용하는가? (사용 목적)

### 보안이 핵심적인 이유다.

VPC를 활용하면 외부에서 직접 접근할 수 없는 독립적인 네트워크 환경을 구성할 수 있어서, 보안적으로 안전하게 리소스(EC2, RDS 등)를 사용할 수 있다.

예를 들어, EC2 인스턴스 2대가 있다고 가정하자. 그런데 1대의 인스턴스는 인터넷에 자유롭게 접근하면서 사용하고 싶고, 나머지 1대는 좀 더 안전하고 비공개로 사용하고 싶을 수 있다. 이럴 때 VPC를 활용하면 된다.

## CIDR (Classless Inter-Domain Routing)이란?

CIDR은 네트워크에서 IP 주소 범위를 표현하는 방식으로, 특히 AWS에서 VPC나 서브넷을 만들 때 반드시 사용하는 표기법이다.

IP 주소를 **유연하고 효율적으로** 표현하고 할당하기 위해 사용되는 표기법

### 형식

IP주소/서브넷 마스크 비트 수

예: 192.168.0.0/24

CIDR: 192.168.0.0/24

| 구성 요소       | 의미                                     |
|-------------|----------------------------------------|
| 192.168.0.0 | 시작 IP 주소                               |
| /24         | 앞에서부터 24비트는 네트워크 식별자, 나머지 8비트는 호스트 식별자 |

👉 즉, 이 CIDR은 \*\*256개의 IP 주소(=2<sup>8</sup>)\*\*를 포함하는 네트워크 범위를 의미합니다.

### ◆ CIDR에 따른 IP 개수

| CIDR | IP 개수  | 설명                           | 📄 |
|------|--------|------------------------------|---|
| /16  | 65,536 | VPC에서 자주 사용 (예: 10.0.0.0/16) |   |
| /24  | 256    | 서브넷에서 자주 사용 (예: 10.0.1.0/24) |   |
| /25  | 128    |                              |   |
| /26  | 64     |                              |   |
| /27  | 32     |                              |   |
| /28  | 16     |                              |   |
| /29  | 8      |                              |   |
| /30  | 4      | 최소 서브넷 중 하나                  |   |

※ 실제로는 네트워크 주소와 브로드캐스트 주소, 그리고 AWS 예약용 IP를 빼면 사용 가능한 IP 수는 더 적습니다.

## 공인 IP (Public IP)란?

전 세계적으로 **고유한 IP** 주소로, 인터넷 상에서 **식별 가능한 주소**이다.

ISP(인터넷 서비스 제공자-KT, LG, SKT 같은 통신사)가 할당하며, 웹사이트, 서버, 클라우드 등 인터넷에 **직접 연결되는 장치**에 사용된다.

```
0.0.0.0 ~ 127.255.255.255
128.0.0.0 ~ 191.255.255.255
192.0.0.0 ~ 233.255.255.255
240.0.0.0 ~ 255.255.255.255
```

## 사설 IP (Private IP)란?

로컬 네트워크(내부 네트워크)에서만 사용되는 IP 주소

인터넷 상에서는 직접 접근 불가

NAT(Network Address Translation)를 통해 외부와 통신 가능

10.0.0.0 ~ 10.255.255.255  
 172.16.0.0 ~ 172.31.255.255  
 192.168.0.0 ~ 192.168.255.255

## 공인 IP와 사설 IP의 차이 요약

| 항목     | 공인 IP                | 사설 IP             |
|--------|----------------------|-------------------|
| 유일성    | 전 세계에서 유일            | 네트워크 내에서만 유일      |
| 인터넷 접근 | 가능                   | 직접 접근 불가 (NAT 필요) |
| 발급     | ISP 또는 클라우드에서 할당     | 로컬 네트워크에서 자동 할당   |
| 비용     | 유료 (공급 한정적)          | 무료                |
| 예시     | 8.8.8.8 (Google DNS) | 192.168.0.1 (공유기) |

## 서브넷(Subnet)이란?

Subnet(서브넷)은 하나의 네트워크(IP 대역=VPC)를 여러 개의 작은 네트워크로 나눈 것으로 즉, IP 주소 범위를 분할해서 내부적으로 목적에 따라 구분하는 것이다.

### 예시로 이해하기

예를 들어 IP 주소 범위가 **192.168.0.0/24**인 경우,

→ 이걸 총 256개(0~255)의 IP를 가질 수 있는 네트워크이다.

이를 두 개의 서브넷으로 나눈다면 ,

- **192.168.0.0/25** (0~127) – [192.168.0.0 ~ 192.168.0.127] 총 128개
- **192.168.0.128/25** (128~255) – [192.168.0.128 ~ 192.0.255] 총 128개

이렇게 하나의 큰 네트워크를 나누어 각각의 목적에 맞게 사용할 수 있다.

### AWS에서 서브넷을 왜 사용하는가?

| 이유                                                                                            | 설명                                        |
|-----------------------------------------------------------------------------------------------|-------------------------------------------|
|  보안 목적       | 퍼블릭 서브넷(인터넷과 연결됨)과 프라이빗 서브넷(내부에서만 사용)을 분리 |
|  인터넷 접근 제어   | 외부에서 접근 가능한 리소스와 내부에서만 접근 가능한 리소스를 나누기 위함 |
|  가용성 분산      | 여러 AZ(가용 영역)에 서브넷을 분산시켜 장애 대비             |
|  네트워크 구조 최적화 | DB, 애플리케이션, 프록시 등을 계층화하여 배치 가능            |
|  확장성과 관리 효율  | 서비스마다 다른 서브넷을 두면 구분이 쉬워지고 관리도 간편해짐        |

### 퍼블릭 vs 프라이빗 서브넷

| 구분     | 퍼블릭 서브넷          | 프라이빗 서브넷           |
|--------|------------------|--------------------|
| 인터넷 접속 | O (인터넷 게이트웨이 연결) | X (직접 연결 없음)       |
| 용도     | 웹 서버, 프록시 서버 등   | DB 서버, 내부 API 서버 등 |
| 보안 수준  | 낮음 (접근 허용)       | 높음 (내부 통신 전용)      |

## 인터넷 게이트웨이(IGW-Internet Gateway)란?

AWS의 인터넷 게이트웨이는 VPC 안의 리소스가 퍼블릭 인터넷과 통신할 수 있도록 연결해주는 장치로 AWS VPC에 연결된 라우팅 가능한 인터넷 출입구 역할을 하는 AWS 네트워크 컴포넌트이다.

즉, VPC 안의 EC2 인스턴스가 인터넷과 통신하려면 반드시 IGW가 필요하다.

### 왜 사용하는가? (사용 목적)

| 목적                                                                                             | 설명                                                 |
|------------------------------------------------------------------------------------------------|----------------------------------------------------|
|  인터넷과의 연결   | 퍼블릭 서브넷의 EC2 인스턴스가 인터넷에서 요청을 받고 응답을 보낼 수 있도록 함     |
|  양방향 통신     | EC2 → 인터넷 (예: 소프트웨어 다운로드)<br>인터넷 → EC2 (예: 웹서버 요청) |
|  퍼블릭 서브넷 정의 | IGW가 연결된 서브넷만 퍼블릭 서브넷으로 간주                         |

## IGW를 쓰기 위한 구성 조건

퍼블릭 인터넷과 통신하려면 다음 세 가지가 모두 설정되어야 합니다:

1. ☒ VPC에 IGW 연결
2. ☒ 서브넷 라우팅 테이블에 IGW를 경유지(타겟)로 지정
3. ☒ EC2 인스턴스에 퍼블릭 IP 또는 탄력적 IP(EIP) 할당

## 구성 예시

```
VPC: 10.0.0.0/16
└─ Subnet: 10.0.1.0/24 (Public Subnet)
    └─ EC2: 10.0.1.10 (퍼블릭 IP 있음)
```

라우팅 테이블:

- 10.0.0.0/16 → local
- 0.0.0.0/0 → igw-12345678

인터넷 게이트웨이:

- VPC에 연결됨

이렇게 설정하면 **10.0.1.10** EC2 인스턴스는 인터넷에서 접속이 가능합니다.

## 정리하면,

| 항목         | 설명                                          |
|------------|---------------------------------------------|
| IGW란?      | VPC를 인터넷과 연결해주는 게이트웨이                       |
| 왜 필요?      | EC2 인스턴스가 인터넷과 통신하려면 반드시 필요                 |
| 퍼블릭 서브넷 조건 | IGW 연결 + 라우팅 설정 + 퍼블릭 IP 보유                 |
| 구성 요소      | VPC에 하나만 연결 가능. 서브넷과 직접 연결하지 않음 (라우팅으로 연결됨) |

## 라우팅 테이블(Routing Table)이란?

VPC 안에서 네트워크 트래픽이 어디로 갈지 결정하는 규칙(경로)의 목록으로  
즉, "이 IP로 가는 요청은 어디로 보내야 할까?" 를 정의하는 네트워크의 길잡이 역할이다.

### 구성 방식

각 라우팅 테이블은 하나 이상의 경로(Route) 항목으로 구성된다.

| 대상(IP 대역)      | 대상 경로(Target)          |
|----------------|------------------------|
| 10.0.0.0/16    | local (VPC 내부 통신용)     |
| 0.0.0.0/0      | igw-xxxxxxx (인터넷 트래픽용) |
| 192.168.1.0/24 | pcx-xxxxxxx (피어링 연결용)  |

### 주요 용도

| 목적                                                                                               | 설명                                |  |
|--------------------------------------------------------------------------------------------------|-----------------------------------|--|
|  인터넷 통신 설정    | 0.0.0.0/0 → IGW 를 추가해서 퍼블릭 서브넷 구성 |  |
|  프라이빗 인터넷 접근  | 0.0.0.0/0 → NAT Gateway 설정        |  |
|  VPC 피어링      | 다른 VPC로 가는 경로 지정                  |  |
|  트래픽 제한 또는 분리 | 서브넷마다 라우팅 테이블을 다르게 설정해서 통신 경로 제어  |  |

### 기본 구성 구조

AWS에서 라우팅 테이블은 다음과 같이 연결

```
[ VPC ]
├─ Subnet A → Routing Table A
├─ Subnet B → Routing Table B
```

- 하나의 서브넷은 하나의 라우팅 테이블만 사용 가능
- 하나의 라우팅 테이블은 여러 서브넷에 연결 가능
- 기본적으로 VPC 생성 시 기본 라우팅 테이블(Default Route Table)이 함께 생성된다.



## 정리

| 항목         | 설명                                                |
|------------|---------------------------------------------------|
| 라우팅 테이블이란? | 네트워크 트래픽의 목적지를 결정하는 규칙 모음                         |
| 기본 경로      | <code>local</code> 경로는 항상 존재 (같은 VPC 내 통신)        |
| 외부 통신      | <code>0.0.0.0/0</code> 경로를 IGW 또는 NAT Gateway로 지정 |
| 서브넷 연결     | 서브넷은 반드시 하나의 라우팅 테이블과 연결됨                         |

## NAT(Network Address Translation)란?

프라이빗 IP를 퍼블릭 IP로 변환하여 내부 네트워크(프라이빗 서브넷)의 인스턴스가 **인터넷에 나갈 수 있게 해주는 기술이다**. 하지만 외부에서 내부로 직접 접근하는 건 차단한다.

## AWS에서 NAT가 필요한 이유

프라이빗 서브넷의 EC2 인스턴스는 :

- 보안상 인터넷에서 **직접 접근을 막고 싶고**
- 소프트웨어 설치, 업데이트, 외부 API 호출 등을 위해 **인터넷으로 나갈 수는 있어야 함**.

이럴 때 사용하는 것이 바로 **NAT Gateway** 또는 **NAT 인스턴스**이다.

## 예시

VPC: 10.0.0.0/16

[ 퍼블릭 서브넷 ]

- └ NAT Gateway (퍼블릭 IP 보유)
  - └ IGW를 통해 인터넷과 연결

[ 프라이빗 서브넷 ]

- └ EC2 (프라이빗 IP만 있음)
  - └ 라우팅 테이블: 0.0.0.0/0 → NAT Gateway

## 보안 그룹(Security Group)이란?

- EC2 인스턴스를 포함한 AWS 자원에 적용되는 **가상 방화벽**
- 인바운드(들어오는 트래픽), 아웃바운드(나가는 트래픽) 규칙을 정의
- 상태 저장(Stateful) : 인바운드가 허용되면, 해당 요청에 대한 응답은 아웃바운드 설정과 무관하게 허용됨

## 인바운드 규칙(Inbound Rule)

외부에서 EC2로 들어오는 트래픽을 허용

| 유형     | 프로토콜 | 포트 범위 | 소스(IP/보안그룹)    | 설명                |
|--------|------|-------|----------------|-------------------|
| SSH    | TCP  | 22    | 203.0.113.0/24 | 특정 IP에서 SSH 접근 허용 |
| HTTP   | TCP  | 80    | 0.0.0.0/0      | 모든 IP에서 웹 접근 허용   |
| HTTPS  | TCP  | 443   | 0.0.0.0/0      | 모든 IP에서 보안 웹 허용   |
| Custom | TCP  | 3306  | 10.0.0.0/16    | 특정 VPC의 DB 접근 허용  |

## 아웃바운드 규칙(Outbound Rule)

EC2에서 외부로 나가는 트래픽을 허용

| 유형     | 프로토콜 | 포트 범위 | 목적지(IP/보안그룹) | 설명                        |
|--------|------|-------|--------------|---------------------------|
| ALL    | ALL  | ALL   | 0.0.0.0/0    | 모든 외부로 나가는 트래픽 허용 (기본 설정) |
| Custom | TCP  | 443   | 0.0.0.0/0    | HTTPS를 통해 외부로 나가기         |

## IAM(Identity and Access Management)이란?

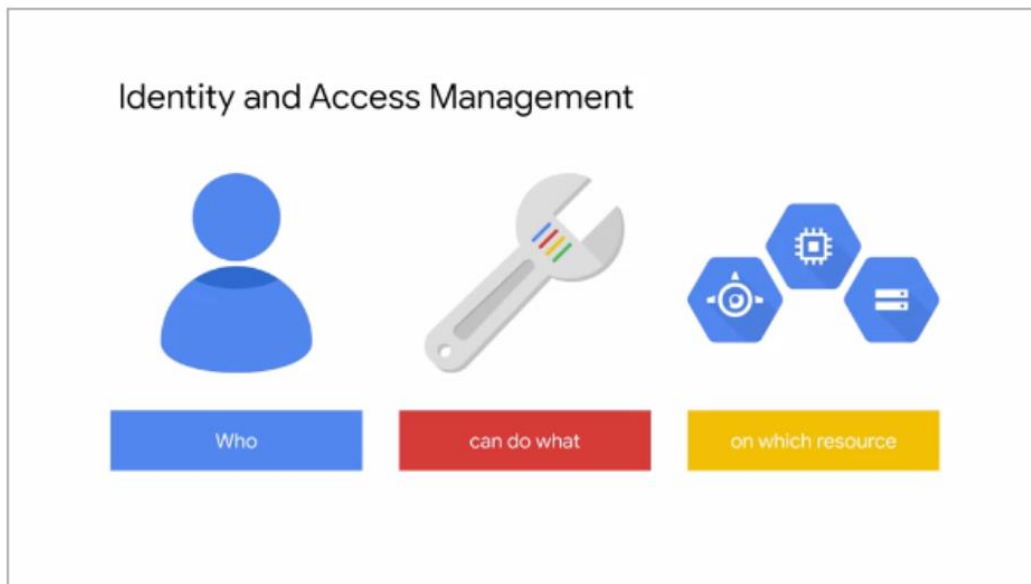
IAM (Identity and Access Management)는 AWS에서 사용자(User), 그룹(Group), 권한(Policy)을 관리하는 서비스이다.



Roles(역할) : AWS 리소스에 의해 사용되며, 역할은 여러 개의 정책 문서를 포함할 수 있음

Policy(정책) : 사용자가 어떤 것을 할 수 있고 어떤 것을 할 수 없는지를 정의하는 문서

즉, "누가 어떤 AWS 자원에 어떤 권한으로 접근할 수 있는가"를 제어하는 보안 시스템이다.



참고: <https://support.bespinglobal.com/ko/support/solutions/articles/73000541479--cloud-iam-iam-%EC%9D%B4%EB%9E%80-%EB%AC%B4%EC%97%B7%EC%9D%B8%EA%B0%80%EC%9A%94->

### 루트 계정과 IAM 계정의 차이

| 항목        | 루트 계정 (Root User) | IAM 사용자               |
|-----------|-------------------|-----------------------|
| 생성 방법     | AWS 가입 시 생성       | 루트 계정 또는 관리자 IAM이 생성  |
| 권한        | 모든 AWS 리소스에 무제한   | 할당된 정책(Policy)에 따라 제한 |
| 사용 권장 여부  | ❌ 사용 최소화 (보안 위험)  | ✅ 실무에서 사용 권장          |
| MFA 설정 가능 | 가능 (강력히 권장)       | 가능                    |

## 왜 IAM 계정이 필요한가?

| 이유                                                                                      | 설명                                                   |
|-----------------------------------------------------------------------------------------|------------------------------------------------------|
|  보안    | 루트 계정은 해킹당하면 AWS 전체가 털림. IAM 사용자에게 필요한 권한만 주면 위험 줄어듦 |
|  팀 협업  | 팀원마다 계정 생성, 각자 권한 설정 가능 (예: 개발자, 관리자, 회계 담당자)        |
|  추적성   | 누가 어떤 작업을 했는지 추적 가능 (CloudTrail과 연동)                 |
|  역할 분리 | 시스템관리자, 개발자, 운영자 등 역할별로 세분화된 권한 관리 가능                |

## IAM 사용자 생성 및 root와 동일 권한 부여 방법

### [1단계] IAM 사용자 생성

1. AWS Management Console 접속 (루트 계정으로 로그인)
2. 서비스 → "IAM" 선택
3. 사용자 → 사용자 생성 클릭
4. 사용자 이름: admin-user 등 설정
5. 액세스 유형 선택
  - ☒ 콘솔 액세스 (웹 로그인 가능)
  - ☒ 프로그래밍 액세스 (CLI/SDK 사용 시 필요)

### [2단계] 권한 설정

6. 권한 부여 방식 선택
  - "기존 정책 직접 연결"
  - AdministratorAccess 정책 체크 ☒
 → root 계정과 동일한 수준의 권한을 부여

이 정책은 모든 AWS 서비스에 대한 전체 액세스를 허용한다.

### [3단계] 태그(선택) → 검토 → 사용자 생성

- 완료 후 콘솔 로그인 URL 확인 가능  
예: <https://<계정ID>.signin.aws.amazon.com/console>

- 생성된 사용자의 비밀번호 설정 링크가 제공됨

#### [4단계] 로그인 테스트

- 새로 만든 IAM 사용자로 로그인 (위 URL + 사용자 이름 + 비밀번호 입력)
- 루트 계정처럼 AWS 콘솔에 접근 가능

#### [5단계] MFA 이중 인증 설정 (강력 추천)

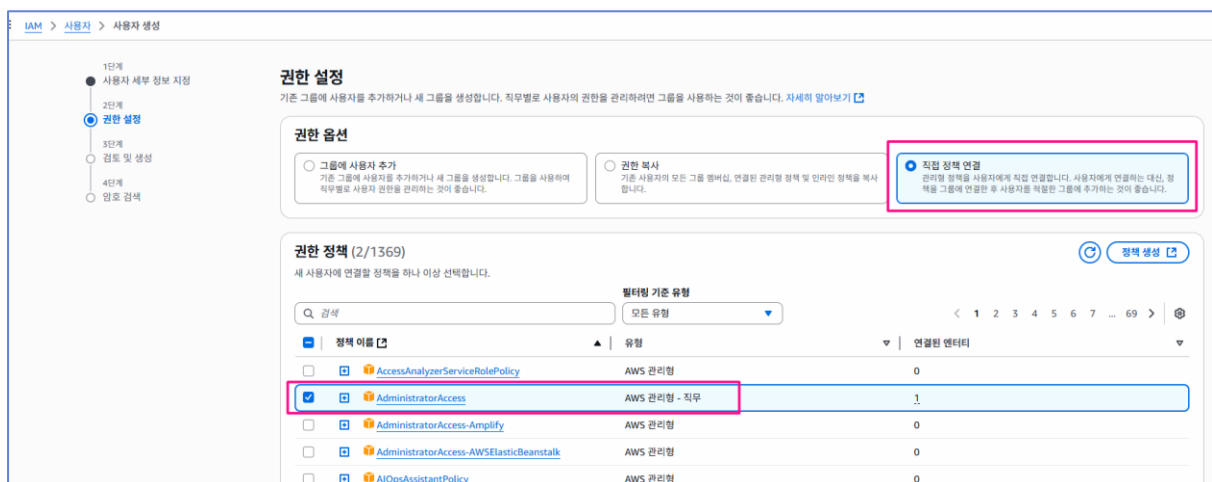
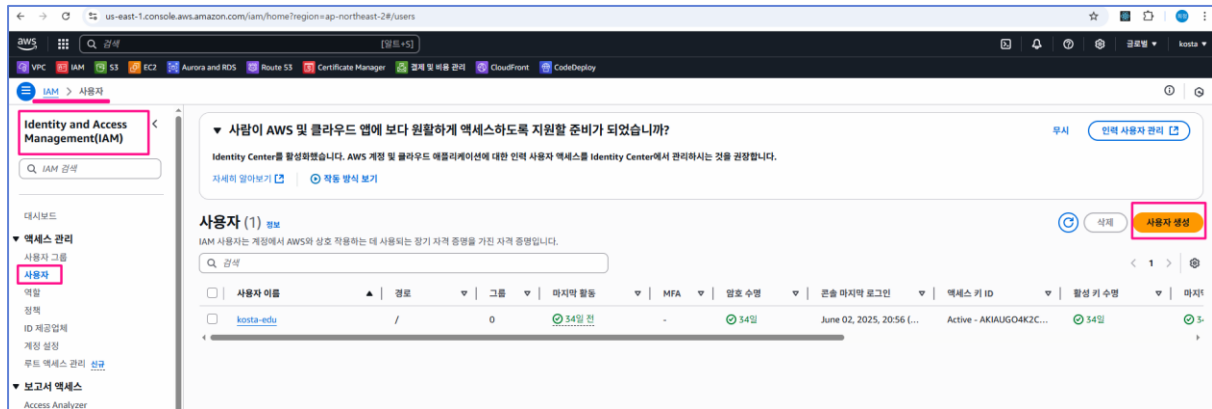
1. IAM 사용자로 로그인
2. 우측 상단 사용자 이름 클릭 → "보안 자격 증명"
3. **MFA 활성화** 선택
4. Authenticator 앱(예: Google Authenticator)과 연동

#### IAM 사용자와 루트 계정의 실무 사용 방식

| 작업 예시                  | 누가 수행해야 하나?      |
|------------------------|------------------|
| 결제 수단 등록, 루트 계정 MFA 설정 | 루트 계정만 가능        |
| EC2, S3, Lambda 생성/관리  | IAM 사용자 가능       |
| IAM 사용자 관리, 권한 부여      | 루트 또는 관리자 IAM 가능 |

#### [IAM 계정 생성 실습]

1. 루트 계정으로 로그인하기
2. IAM 메뉴로 이동 후 사용자 생성하기
3. 사용자 이름 입력하고 management console로 로그인 할 수 있도록 체크박스 체크/IAM 사용자 생성/자동생성된 암호 사용하려면 선택하고 다음 클릭



IAM > 사용자 > 사용자 생성

1단계 사용자 세부 정보 지정  
2단계 **권한 설정**  
3단계 검토 및 생성  
4단계 암호 검색  
완료 검색

### 권한 설정

기본 그룹에 사용자를 추가하거나 새 그룹을 생성합니다. 직무별로 사용자의 권한을 관리하려면 그룹을 사용하는 것이 좋습니다. [자세히 알아보기](#)

**권한 옵션**

- ☐ 그룹에 사용자 추가  
기본 그룹에 사용자를 추가하거나 새 그룹을 생성합니다. 그룹을 사용하여 직무별로 사용자 권한을 관리하는 것이 좋습니다.
- ☐ 권한 복사  
기본 사용자의 모든 그룹 멤버십, 연결된 관리형 정책 및 인라인 정책을 복사합니다.
- ☒ 직접 정책 연결  
관리형 정책을 사용자에게 직접 연결합니다. 사용자에게 연결하는 대신, 정책을 그룹에 연결한 후 사용자를 적절한 그룹에 추가하는 것이 좋습니다.

**권한 정책 (2/1369)**  
새 사용자에게 연결할 정책을 하나 이상 선택합니다.

필터링 기준 유형: 모든 유형 | 17 개 일치

검색: s3

| 정책 이름                                                            | 유형             | 연결된 엔티티 |
|------------------------------------------------------------------|----------------|---------|
| <input type="checkbox"/> AmazonDMSRedshiftFS3Role                | AWS 관리형        | 0       |
| <input checked="" type="checkbox"/> <b>AmazonS3FullAccess</b>    | <b>AWS 관리형</b> | 0       |
| <input type="checkbox"/> AmazonS3ObjectLambdaExecutionRolePolicy | AWS 관리형        | 0       |
| <input type="checkbox"/> AmazonS3OutpostsFullAccess              | AWS 관리형        | 0       |

VPC IAM S3 ECR Aurora and RDS Route 53 Certificate Manager 검색 및 비유 CloudFront CodeDeploy

IAM > 사용자 > 사용자 생성

1단계 사용자 세부 정보 지정  
2단계 권한 설정  
3단계 검토 및 생성  
4단계 암호 검색  
완료 검색

### 검토 및 생성

선택 사항을 검토합니다. 사용자를 생성한 후 자동 생성된 암호를 보고 다운로드할 수 있습니다(활성화된 경우).

**사용자 세부 정보**

사용자 이름: kosta-test | 권한 암호 유형: Autogenerated | 암호 재설정 필요: 예

**권한 요약**

| 이름                    | 유형           | 다음과 같이 사용 |
|-----------------------|--------------|-----------|
| AdministratorAccess   | AWS 관리형 - 직무 | 권한 정책     |
| AmazonS3FullAccess    | AWS 관리형      | 권한 정책     |
| IAMUserChangePassword | AWS 관리형      | 권한 정책     |

**태그 - 선택 사항**  
태그는 리소스를 식별, 구성 또는 검색하는 데 도움이 되도록 AWS 리소스에 추가할 수 있는 키 값 페어입니다. 이 사용자와 연결할 태그를 선택합니다. 리소스와 연결된 태그가 없습니다.

[새 태그 추가](#)  
최대 50개의 태그를 더 추가할 수 있습니다.

취소 [이전](#) **사용자 생성**

IAM > 사용자 > 사용자 생성

**사용자가 성공적으로 생성됨**  
AWS Management Console에 로그인하기 위한 사용자의 암호와 이메일 지침을 보고 다운로드할 수 있습니다. [사용자 보기](#)

**암호 검색**  
아래에서 사용자의 암호를 보고 다운로드하거나 AWS 콘솔에 로그인하기 위한 사용자 지침을 이메일로 보낼 수 있습니다. 지금이 이 암호를 확인 및 다운로드할 수 있는 유일한 시간입니다.

**콘솔 로그인 세부 정보**

콘솔 로그인 URL: <https://288761761976.signin.aws.amazon.com/console>

사용자 이름: kosta-test

콘솔 암호: 8l-vH4je [숨기기](#)

[이메일 로그인 지침](#)

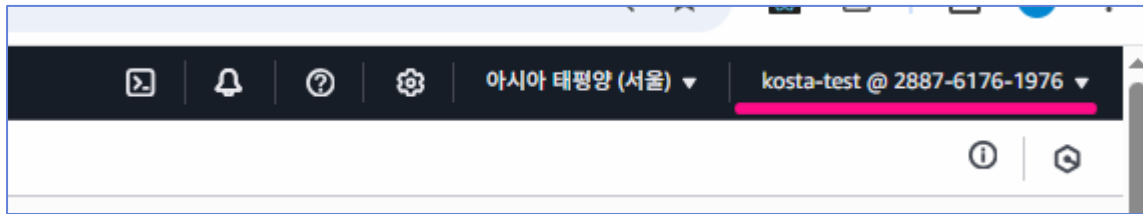
취소 [.csv 파일 다운로드](#) [사용자 목록으로 돌아가기](#)

**.csv를 다운로드하면 아이디와 비밀번호가 들어있다**

## 접속하면

메모장에  
계정ID 12자리  
사용자이름  
비밀번호  
저장해 놓는다.

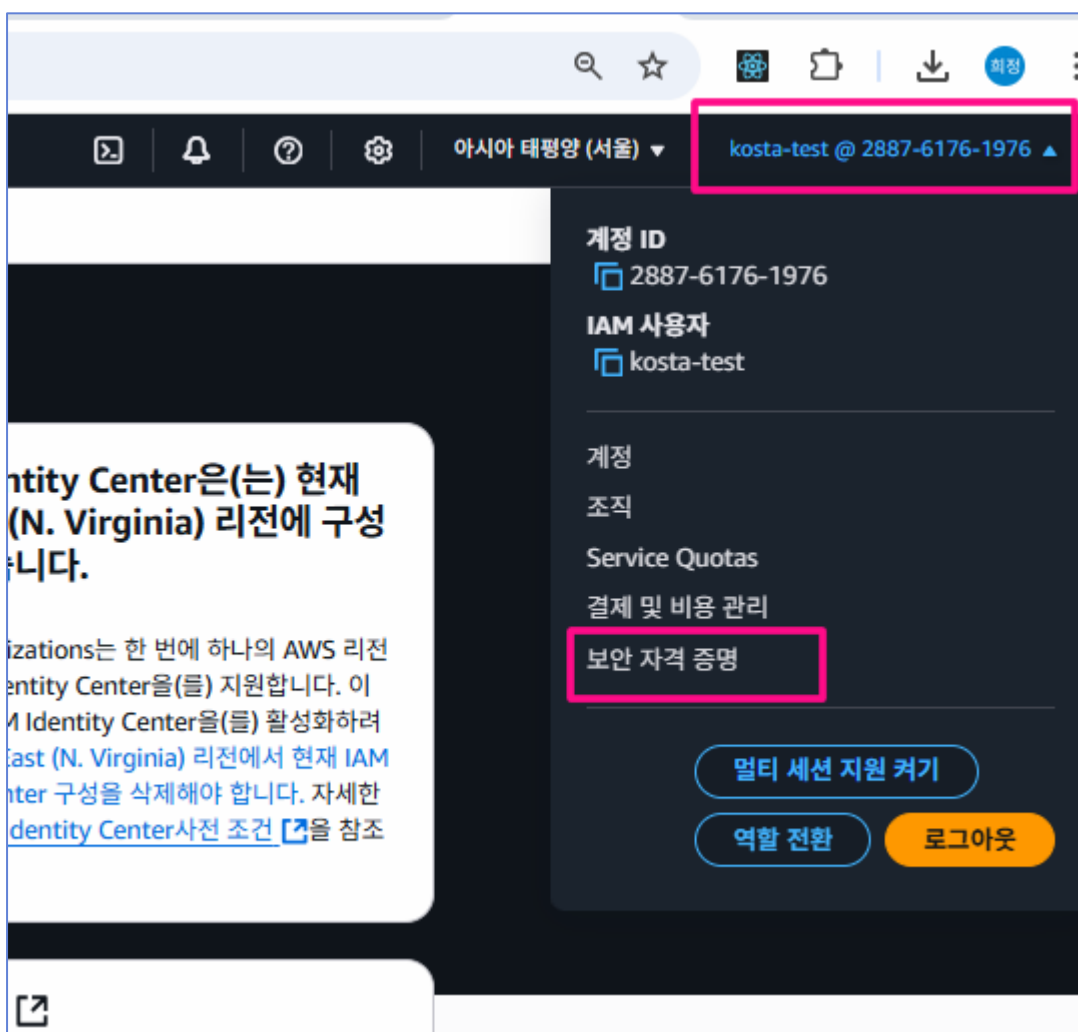




### 엑세스키 생성하기(AWS CLI , AWS API 사용시 필요)

Aws를 접근할 수 있는 권한이 필요하므로 우리의 iam 계정에 보안자격증명으로 권한을 부여받아야 한다.

계정 클릭하고 보안자격증명 선택한 후 엑세스키 생성하고  
csv 다운로드 받아놓기



IAM > 보안 자격 증명 > 액세스 키 만들기

1단계

액세스 키 모범 사례 및 대안

2단계 - 선택 사항

설명 태그 설정

3단계

액세스 키 검색

설명 태그 설정 - 선택 사항

이 액세스 키에 대한 설명은 이 사용자에게 태그로 연결되고, 액세스 키와 함께 표시됩니다.

설명 태그 값

이 액세스 키의 용도와 사용 위치를 설명합니다. 좋은 설명은 나중에 이 액세스 키를 자신있게 교체하는 데 유용합니다.

최대 256자까지 가능합니다. 허용되는 문자는 문자, 숫자, UTF-8로 표현할 수 있는 공백 및 \_ : / = + . @입니다.

취소

이전

액세스 키 만들기

1단계  
2단계 - 신규 사용  
3단계  
● 액세스 키 검색

### 액세스 키 검색

**액세스 키**  
문실하거나 잊어버린 비밀 액세스 키는 검색할 수 없습니다. 대신 새 액세스 키를 생성하고 이전 키를 비활성화합니다.

| 액세스 키                | 비밀 액세스 키 |
|----------------------|----------|
| AKIAUGO4K2C4MYKBYNFX | ***** 표시 |

**액세스 키 모범 사례**

- 액세스 키를 일반 텍스트, 코드 리포지토리 또는 코드로 저장해서는 안 됩니다.
- 다 이상 필요 또는 경우 액세스 키를 비활성화하거나 삭제합니다.
- 최소 권한을 할당합니다.
- 액세스 키를 정기적으로 교체합니다.

액세스 키 관리에 대한 자세한 내용은 [AWS 액세스 키 관리 모범 사례](#)를 참조하세요.

[.csv 파일 다운로드](#) [완료](#)

클릭